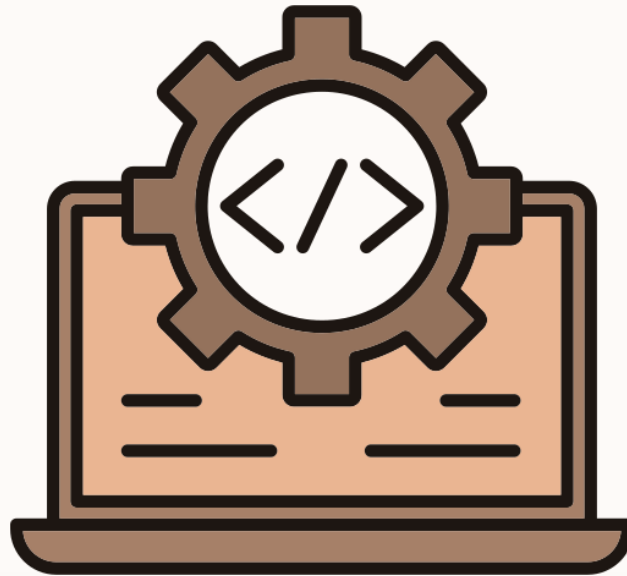




مشروع

المترجمات 2



إعداد الطلاب

- محمد علي حسن النعيمي /فئة 12/
- كريم رأفت البزرة /فئة 12/
- أسامة محمد سامر زرقاوي /فئة 12/
- نور الدين كساب خليل /فئة 12/
- محمد الحسن محمود زعيتر /فئة 12/

3	1: شرح ملف (Parser)
8	2: مخطط AST
9	3: التحليل الدلالي (Semantic Analysis)
9	3.1 الفحوصات على جانب Python و Flask
10	3.2 الفحوصات على جانب HTML
10	3.3 الفحوصات على جانب Jinja و CSS
11	4: مرحلة التوليد (Code Generation)
11	4.1 المخطط العام لمسار الترجمة
11	4.2 مكوّنات مرحلة التوليد
12	4.3 جانب Python: من الشيفرة إلى القيم
13	4.4 جانب Jinja: من الشجرة إلى HTML
14	4.5 إعادة التوليد عند تغيّر البيانات
15	5: ملفات الخرج
15	6: نتائج التشغيل الفعلي

Python - Flask	
القاعدة (Rule)	الشرح (Explanation)
program	هذه هي القاعدة الرئيسية ونقطة بداية التحليل. تنص على أن البرنامج يتكون من صفر أو أكثر من الجمل (statement) وينتهي بنهاية الملف (EOF).
statement	الجمل هي الوحدات الأساسية التي يتكون منها البرنامج، ويمكن أن تكون إما جمل بسيطة (simpleStatement) تكتفي بسطر واحد أو جملة مركبة (compoundStatement) تشمل كتلة برمجية.
simpleStatement	جملة تكتب في سطر واحد. تشمل الإسناد، الاستيراد، return, break, continue, pass, global أو تعبير ينتهي بسطر جديد.
compoundStatement	جملة مركبة تتطلب كتلة برمجية (مجموعة جمل مُعرّفة بالمسافة البادئة). تشمل if, for, while، تعريف الدوال والفئات.
returnStatement	جملة لإرجاع قيمة من دالة، تبدأ بـ return يتبعها اختيارياً تعبير أو قائمة تعابير.
breakStatement	جملة للخروج من حلقة while أو for فوراً، تتكون من كلمة مفتاحية break
continueStatement	جملة لتخطي بقية التكرار الحالي في حلقة for أو while والانتقال إلى التكرار التالي، تتكون من continue.
importStatement	جملة لاستيراد وحدات (modules) لها شكلان: Import module1, module2 From module import item1, item2
importItem	يمثل عنصراً يتم استيراده، يمكن أن يكون له اسم مستعار باستخدام as، مثال: numpy as np
importModule	يمثل مسار الوحدة المستوردة، قد يحتوي على نقاط للإشارة إلى الوحدات الفرعية، مثال: os.path
globalStatement	تعلن أن متغيراً أو أكثر متغيرات عامة (global) داخل نطاق الدالة.
passStatement	جملة فارغة لا تفعل شيئاً، تستخدم كعنصر نائب حيثما كانت الجملة مطلوبة نحوياً.
classDef	لتعريف فئة (class)، تبدأ بـ class، يتبعها اسم الفئة، اختيارياً الفئات التي ترثها بين قوسين، ثم نقطتين وكتلة برمجية.
body	يمثل كتلة برمجية، يتكون من مسافة بادئة (INDENT)، جملة أو أكثر، ثم إلغاء المسافة البادئة (DEDENT).
funcdef	لتعريف الدالة، يمكن أن يحتوي على decorators، يبدأ بـ def، يتبعه اسم الدالة، معاملات بين قوسين، اختيارياً نوع الإرجاع، ثم نقطتين وكتلة برمجية.

decorators	واحد أو أكثر من ال decorators التي تسبق تعريف الدالة أو ال class.
decorator	يبدأ ب @، يتبعه اسم (مثل app.route)، واختيارياً وسيطات بين قوسين.
dottedName	اسم معرف يمكن أن يحتوي على نقاط، للإشارة إلى سمات داخل وحدات أو فئات، مثال: flask.request
parameters	قائمة من المعاملات (param) التي تأخذها الدالة، مفصولة بفواصل.
param	معامل واحد في تعريف الدالة، يمكن أن يكون معاملاً عادياً (y=10 أو x)، أو متغير الطول (*args)، أو كلمات مفتاحية متغيرة الطول (**kwargs).
forStatement	حلقة تكرارية، تكرر على كل عنصر في تسلسل (مثل قائمة).
whileStatement	حلقة تكرارية تستمر طالما أن الشرط المحدد صحيح (True)
ifStatement	جملة شرطية. تنفذ كتلة برمجية إذا كان الشرط صحيحاً، مع إمكانية وجود كتل إضافية elif و else
assignmentStatement	جملة إسناد قيمة لمتغير، يمكن أن تكون إسناداً بسيطاً (=) أو مختصراً (+=, -=)
targetList	قائمة من المتغيرات التي سيتم إسناد القيم إليها، مفصولة بفواصل
target	الهدف الذي يتم إسناد القيمة إليه، يمكن أن يكون متغيراً بسيطاً، عنصراً في قائمة (list[i]) أو سمة في كائن (obj.attr).
expressionList	قائمة من التعابير (expression) مفصولة بفواصل.
expression	التعبير الرئيسي الذي يحدد أسبقية العمليات، يشمل الحسابات، المقارنات، والعمليات المنطقية (and, or, not).
augmentedAssignment	عوامل الإسناد المختصرة مثل +=, -=, /=, *=
atomExpression	التعبير الأساسي إما قيمة أولية (atom) أو اسم متغير يتبعه لاحقات مثل استدعاء دالة () أو فهرسة [] أو وصول لسمة ..
Trailer	لاحقة تتبع اسماً. يمكن أن تكون استدعاء دالة (args)، فهرسة [index]، أو وصول لسمة .attr
atom	أصغر وحدة في ال expression. يمكن أن تكون قيمة حرفية (رقم، نص)، قائمة، قاموس، أو تعبيراً بين قوسين.
keyValueList	قائمة من أزواج المفتاح والقيمة () مفصولة بفواصل، تستخدم لتعريف القواميس.
keyValue	زوج مفتاح وقيمة واحد داخل قاموس.
Literal	يمثل قيمة حرفية ثابتة مثل رقم (123)، نص ("hello")، قيمة منطقية (True, False)، أو None.
arglist	قائمة من الوسائط (argument) التي يتم تمريرها إلى دالة عند استدعائها.
argument	وسيلة واحدة لاستدعاء دالة. يمكن أن تكون قيمة، أو زوجاً من الكلمة المفتاحية والقيمة (name = value) أو قاموساً (**dict).

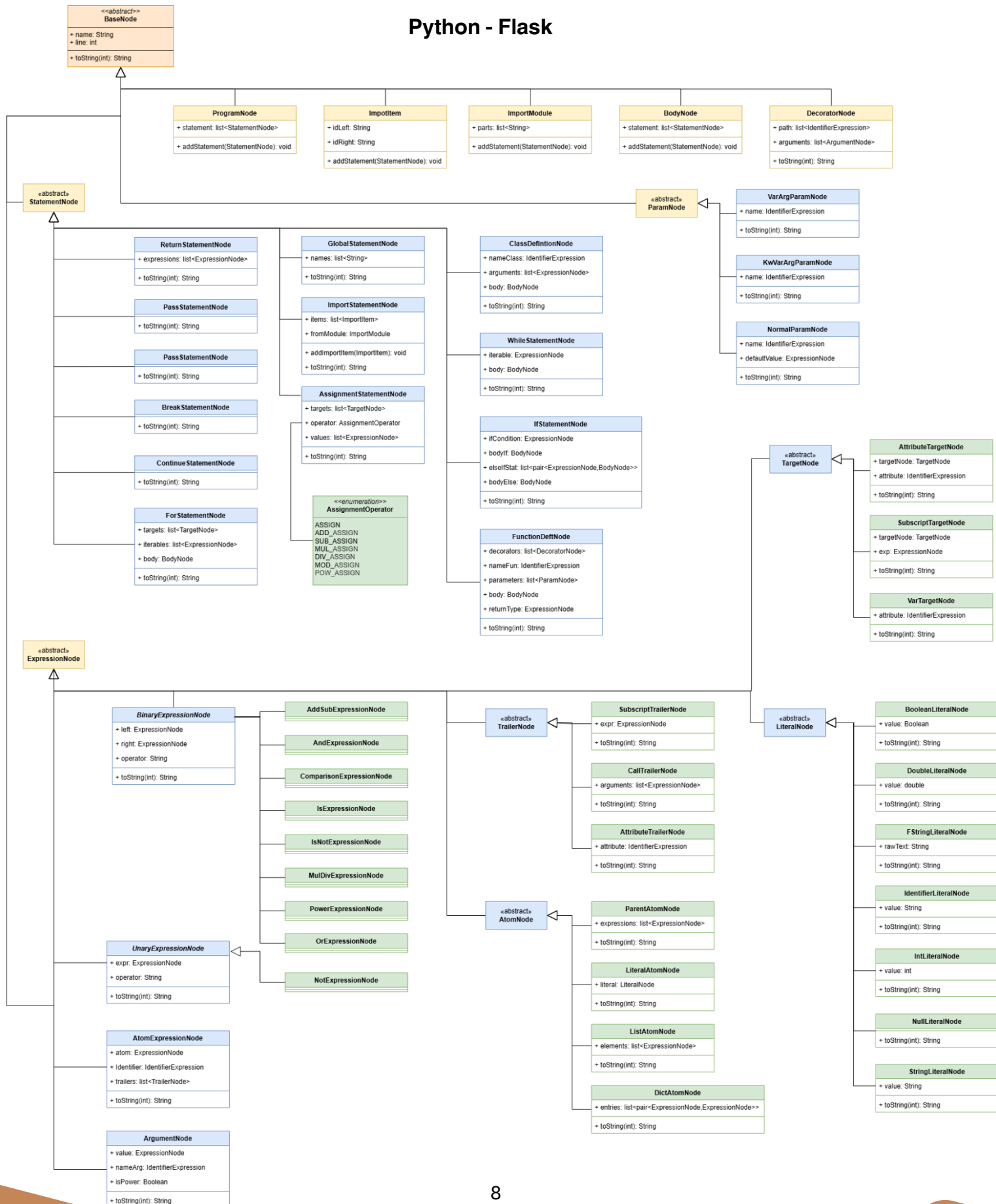
القاعدة (Rule)	الشرح (Explanation)
htmlDocument	القاعدة الجذرية للملف، مساحات بيضاء اختيارية، Data Type Document اختيارية، محتوى HTML، وتنتهي بنهاية الملف.
htmlElements	تجميع لعناصر HTML محاظًا بمحتويات أو عناصر فرعية اختيارية.
htmlElement	تعريف عنصر HTML: إما tag، أو style element.
htmlContent	المحتوى داخل عنصر HTML قد يتضمن نص، عناصر فرعية، تعليقات HTML أو كتل Jinja.
htmlAttribute	tag attributes: اسم سمة واختيارياً قيمة مساوية.
HtmlCharData	بيانات نصية داخل HTML (نص حقيقي).
htmlMisc	محتويات متعددة غير أساسية داخل HTML، مثل التعليقات أو المساحات البيضاء.
htmlComment	تعليق HTML الكامل.
jinjaBlock	أي كتلة Jinja2 داخل HTML: تعليق، تعبير أو جملة.
jinjaExpression	تعبير Jinja2 المحاط بعلامتي الفتح والإغلاق. ({{ ... }})
jinjaExprContent	محتوى التعبير الذي يتكون من واحد أو أكثر من التعبيرات الفرعية.
jinjaExprExpression	البناء الرياضي/المنطقي داخل تعابير Jinja (العمليات والمقارنات).
jinjaExprAtomExpression	عدة عناصر من ال Atom نسخة ال Expression
jinjaExprTrailer	لاحقة تتبع اسماً. يمكن أن تكون استدعاء دالة (args)، فهرسة [index]، أو وصول لسمة attr.
jinjaExprAtom	أصغر وحدة في ال expression. يمكن أن تكون قيمة حرفية (رقم، نص)، قائمة، قاموس، أو تعبيراً بين قوسين.
jinjaExprArgument	قائمة من الوسائط (argument) التي يتم تمريرها إلى دالة عند استدعائها.
jinjaStatement	كتل التحكم في Jinja لمثل for و if.
jinjaForBlock	بنية جملة for.
jinjaForStatement	بنية تعليمة for.

بنية جملة if.	jinjalBlock
البناء الرياضي/ المنطقي داخل تعابير Jinja العمليات والمقارنات).	jinjaStmtExpression
عدة عناصر من ال Atom نسخة ال Statement	jinjaStmtAtomExpression
لاحقة تتبع اسماً. يمكن أن تكون استدعاء دالة (args)، فهرسة [index]، أو وصول لسمة attr.	jinjaStmtTrailer
أصغر وحدة في ال expression. يمكن أن تكون قيمة حرفية (رقم، نص)، قائمة، قاموس، أو تعبيراً بين قوسين.	jinjaStmtAtom
القيم الحرفية المستخدمة داخل جمل Jinja.	jinjaStmtLiteral
قائمة من الوسائط (argument) التي يتم تمريرها إلى دالة عند استدعائها.	jinjaStmtArgument
المحتوى المسموح داخل بنية القوالب: نص، عناصر HTML ، تعليقات، وكتل Jinja .	templateContent
قاعدة البدء ل tag ال style	style
مجموع قواعد CSS داخل وسم style .	stylesheet
قاعدة @import في CSS مع أو بدون media queries ونقطة فاصلة.	imports
تعريف @media (استعلام وسائط) يحتوي على قائمة شروط وجسم قواعد.	media
قائمة استعلامات الوسائط مفصولة بفواصل.	mediaQueryList
استعلام وسائط مفرد يحدد نوع/عوامل (مثل screen and (max-width: 600px)).	mediaQuery
تعبير داخل قوس في media مثل (max-width: 600px).	mediaExpression
مجموعة محددات CSS مفصولة بفواصل لقاعدة واحدة.	selectorGroup
محدد مفرد قد يتضمن سلاسل مركبة ومحددات فرعية.	selector
المعامل بين محددات (مثل ~, >, + أو فراغ) لتحديد العلاقة.	combinator
تتابع بسيط من محددات مثل اسم العنصر مع class/ID/pseudo.	simpleSelectorSequence
تحديد نوع عنصر مثل div أو span مع فضاء أسماء (namespace) اختياري.	typeSelector
بادئة مساحة الاسم قبل اسم النوع.	typeNameSpacePrefix

universal	محدد عام '*' مع خيار بادئة namespace.
className	محدد class يسبق بنقطة متبوع باسم معرف.
pseudo	pseudo-class أو pseudo-element تبدأ بنقطتين ':' أو '::'.
functionalPseudo	pseudo التي تأخذ قيمة/معاملًا داخليًا مثل: nth-child().
expression	تعبير يستخدم في قيم CSS يتكون من أرقام، أبعاد، كلمات...
operator_	فواصل أو عوامل مثل '/' أو ';' أو مسافات تفصل بين المصطلحات.
property_	اسم الخاصية أو متغير CSS قبل النقطتين.
ruleset	مجموعة قواعد: محدد/محددات مع جسم خصائص بين { }.
declarationList	قائمة التصريحات داخل جسم القاعدة مفصولة بفواصل وسمي كولن.
declaration	تصريح واحد يربط property بقيمة expr وقد يتضمن !important.
expr	تعبير قيمة يمكن أن يتضمن عدة مصطلحات وعوامل.
term	واحد من أنواع المصطلحات في expr: رقم، نسبة، بُعد، سلسلة، معرف، إلخ.
number	تمثيل رقم مطلق مع إشارة اختيارية.
percentage	قيمة نسبة مع إشارة اختيارية.
dimension	قيمة بُعد مع وحدة مثل px, em.
nestedStatement	قواعد متداخلة داخل CSS (قواعد أو media).
groupRuleBody	جسم مجموعة قواعد مثل { ... } @media.
url	تمثيل قيمة url() في CSS، مقتبسة أو غير مقتبسة.
calcSum	مجموع داخل calc(...) يتألف من منتجات ومصطلحات مع عمليات +/.
calcProduct	جزء الضرب/قسمة في calc.
calcValue	قيمة أساسية في calc: رقم، بُعد، نسبة أو تعبير مُجمّع.
cssIdent	مُعَرَّف CSS عام: اسم أو كلمات محجوزة مثل 'not' أو 'and'.

2: مخطط AST

Python - Flask



مرفع بملف خارجي مع الایمیل

3: التحليل الدلالي (Semantic Analysis)

التحليل الدلالي هو المرحلة التي تلي بناء الشجرة (AST) وتسبق مرحلة التوليد. مهمتها اكتشاف الأخطاء التي يكون البرنامج فيها صحيحاً نحوياً لكنه غير صحيح معنوياً، مثل استعمال متغير غير مُعرّف، أو إغلاق وسم HTML بوسم مختلف. ويعتمد التحليل الدلالي على جدول الرموز (Symbol Table) لأنه يتعامل مع التصريحات والأنواع، بخلاف مرحلة التوليد التي تتعامل مع القيم.

بنية الخطأ الدلالي الموحدة

جميع النتائج الدلالية في المشروع، على جانب Python وعلى جانب HTML و CSS و Jinja، تُمثّل بصنف واحد هو SemanticError، ما يسمح بجمعها وترتيبها وطباعتها في تقرير واحد:

```
public class SemanticError {
    public enum Severity { ERROR, WARNING }

    public final String phase;      // Python | HTML | CSS | Jinja
    public final String message;
    public final int line;
    public final Severity severity;
}
```

الفرق بين النوعين: ERROR خطأ يمنع صحة البرنامج، و WARNING ملاحظة لا تمنع التوليد لكنها تدلّ على مشكلة محتملة، مثل صنف CSS مستعمل وغير مُعرّف.

3.1 الفحوصات على جانب Python و Flask

الفحص	النوع	الشرح
Use of undefined variable	ERROR	استعمال متغير لم يُصرّح عنه في أي نطاق مرئي. يجري البحث في سلسلة النطاقات من الداخل إلى الخارج.
Duplicate parameter	ERROR	تكرار اسم وسيط داخل تعريف الدالة نفسها، مثل <code>def f(x, x)</code> .
Duplicate Class & Function	ERROR	تعريف دالة أو صف باسم موجود مسبقاً في نفس الـ <code>scope</code> .
Type Error	ERROR	تطبيق عملية حسابية بين نوعين غير متوافقين وكلاهما معروفين. مثال: <code>"hello" + 5</code>
Type Mismatch	ERROR	إعلان دالة بـ <code>return type annotation</code> صريح (<code>int / -> str</code>) ثم إرجاع قيمة من نوع مختلف. مثال: <code>def foo() -> int: "return "hello"</code>

جملة return خارج جسم أي دالة. يُتَّبَع ذلك بعدد لعمق الدوال أثناء المرور على الشجرة.	ERROR	return used outside of a function
جملة break خارج حلقة for أو while، ويُتَّبَع بعدد لعمق الحلقات.	ERROR	break used outside of a loop
جملة continue خارج أي حلقة، بالالية نفسها.	ERROR	continue used outside of a loop
قسمة على صفر حربي يمكن كشفها ساكناً قبل التنفيذ، مثل $x / 5 = 0$.	ERROR	Division by zero

3.2 الفحوصات على جانب HTML

الشرح	النوع	الفحص
تكرار قيمة id في المستند نفسه. تُخزّن المعرّفات المشاهدة في مجموعة، وأي إضافة فاشلة تعني تكراراً.	ERROR	Duplicate id
عنصر غير ذاتي الإغلاق ولا يملك وسم إغلاق. يُكتشف من قيمة endTagName حين تساوي No End Tag.	ERROR	Missing closing tag
الوسم مفتوح باسم ومغلق باسم آخر، مثل div مغلق بـ span. تُقارن أسماء وسمي الفتح والإغلاق.	ERROR	Mismatched closing tag

3.3 الفحوصات على جانب CSS و Jinja

الشرح	النوع	الفحص
صنف مستعمل في السمة class لكنه غير معرّف في أي قاعدة CSS. تُجمّع الأصناف المعرّفة أثناء بناء جدول رموز CSS.	WARNING	Class used but not defined
متغيّر مستعمل داخل تعبير أو تعليمة Jinja وغير موجود في السياق. الفحص واعٍ بالنطاقات.	WARNING	Variable is not defined

النطاقات في Jinja

عند الدخول إلى حلقة for يُفتح نطاق جديد يُعرّف فيه متغيّر الحلقة، ويُغلق النطاق عند endfor. لذلك فإن استعمال متغيّر الحلقة بعد نهايتها يُنتج تحذيراً، بينما استعماله داخلها لا يُنتج شيئاً. أما المتغيّرات القادمة من Flask عبر render_template فتُعرّف مسبقاً في النطاق العام، ويجري استخراج أسمائها من خطة التوليد لا من التخمين.

4: مرحلة التوليد (Code Generation)

هي التي تحلّ محلّ الدالة `render_template` الخاصة بـ Flask، لكن وقت الترجمة بدل وقت التشغيل. والنتيجة صفحات HTML جاهزة لا تحتاج إلى سيرفر.

المبدأ الأساسي

التوليد دالة تأخذ عقدة من شجرة القالب وبيئة قيم، وتُعيد نصّاً.

```
render( AST_node , Scope ) -> String
```

والشيء الوحيد الذي يتغيّر أثناء المرور على الشجرة هو الـ `Scope`، وهذا هو سرّ عمل حلقة `for`.

الفرق الجوهرى عن التحليل الدلالي

الوجه	التحليل الدلالي	مرحلة التوليد
ماذا يُخزّن	الأنواع، مثل: <code>products</code> هو <code>list</code>	القيم، مثل: <code>products</code> تساوي ثلاثة عناصر بقيمها
البنية المستعملة	جدول الرموز <code>SymbolTable</code>	بيئة القيم <code>Scope</code>
الهدف	كشف الأخطاء	إنتاج الخرج

لهذا السبب لا تعتمد مرحلة التوليد على جدول الرموز إطلاقاً

4.1 المخطط العام لمسار الترجمة

[1] Python front-end	lex -> parse -> AST -> symbols -> semantic
[2] Python generation	evaluate globals, unroll routes -> render plan
[3] Jinja front-end	parse each template once -> AST -> semantic
[4] Jinja generation	one HTML file per entry of the render plan
[5] Companion files	static and app.py copied verbatim
[6] Compiler reports	compiler_output json and txt files

4.2 مكونات مرحلة التوليد

المكوّن	الدور
Scope	بيئة القيم أثناء التوليد. متداخلة عبر مرجع إلى الأب، فكل دورة من حلقة <code>for</code> تفتح نطاقاً ابناً يحمل متغيّر الحلقة ويختفي بانتهائها.

PythonContextGenerator	جانب Python من التوليد، وهو ثلاث طبقات: مُقيّم تعابير يحوّل عقدة إلى قيمة Java، ومنفّذ تعليمات يشغّل جسم دالة المسار، ومُعَدّد مسارات يقرأ app.route ويفتح وسائط ال URL.
JinjaEvaluator	يقيم تعابير Jinja مقابل ال Scope. يعالج أصناف التعابير في الشجرة، ويطبق قواعد الخاصة ب Python.
HtmlGenerator	يمرّ على شجرة HTML و Jinja وينتج النص: يوسّع حلقة for، ويختار فرعاً واحداً من if، ويستبدل التعابير بقيمها، ويعيد بناء الوسوم والسمات بتنسيق مقروء.
RouteMap	يحوّل مسارات Flask إلى الملفات التي ينتجها المترجم. مسارات التنقل تصبح روابط ملفات، ومسارات الإرسال تُعطل ما لم يكن سيرفر المعاينة يعمل.
AstJsonWriter	يحوّل أي شجرة إلى JSON بالانعكاس على الحقول العامة، فيغطّي شجرة Python وشجرة Jinja بالكود نفسه.
PreviewServer	سيرفر مبني على HttpServer من ال JDK. يخدم الصفحات المولّدة ويستقبل نماذج الإضافة والحذف، ويطلب إعادة التوليد قبل إرجاع المتصفح.

4.3 جانب Python: من الشيفرة إلى القيم

الخطوة الأولى هي حساب القيم الفعلية للمتغيّرات على مستوى الوحدة. فالمُقيّم يمشي على الشجرة ويحوّل كل عقدة تعبير إلى قيمة Java مقابلة:

نوع Python	المقابل في Java
list	List<Object>
dict	Map<String, Object>
str	String
int	Integer
float	Double
None	null

لا يكفي تقييم المتغيرات العامة، لأن السياق الممرّر إلى القالب قد يعتمد على متغير محلي. ففي المسار التالي تعتمد قيمة `product` على وسيط ال URL:

```
@app.route('/product/<int:product_id>')
def product_details(product_id):
    product = None
    for p in products:
        if p['id'] == product_id:
            product = p
            break
    return render_template('product_details.html', product=product)
```

لذلك بُني منفذ تعليمات مصغّر ينفذ جسم الدالة فعلياً: الإسناد، و `if` و `elif` و `else`، و `for`، و `while`، و `break`، و `continue`، و `return`. ويُستعمل نوع خاص لنقل القفزات إلى الأعلى:

```
private enum Flow { NORMAL, BREAK, CONTINUE, RETURN }
private record Signal(Flow flow, ExpressionNode returned) { ... }
```

وبعد انتهاء التنفيذ يُقيّم وسيط `render_template` مقابل النطاق المحلي لا العام، وهذا بالضبط ما يجعل المتغيرات المحلية تُحسب بقيمتها الصحيحة. وحمايةً من التعليق، وُضع سقف لعدد دورات حلقة `while` قدره عشرة آلاف دورة، ويُسجّل تحذير عند تجاوزه.

فتح المسارات

في Flask تُنفذ `render_template` مرة واحدة لكل طلب HTTP. أما المترجم فلا طلبات لديه، لذا عليه أن يُعدّ كل الطلبات الممكنة وقت الترجمة. فالمسار الذي يحمل وسيطاً في عنوانه مع ثلاثة منتجات يعني ثلاث صفحات ممكنة بالضبط. وهذا نظير فتح الحلقات ولكن على مستوى الصفحات.

وقيم الوسائط لا يمكن استنتاجها من الشيفرة، فتُشتقّ من البيانات نفسها: يبحث المترجم عن قائمة على مستوى الوحدة تحمل عناصرها الحقل المطلوب فيأخذ منها القيم. وبهذا لا يُكتب اسم متغير واحد داخل المترجم، ويبقى قابلاً للعمل على أي مشروع Flask آخر.

4.4 جانب Jinja: من الشجرة إلى HTML

يمرّ المولّد على الشجرة ويتعامل مع كل نوع عقدة على حدة:

عقدة الشجرة	ما يفعله المولّد
HtmlTextNode	يُنسخ النص كما هو.
HtmlElementNode	يُعاد بناء الوسم وسماته وأبنائه. ويُنشئ وسم الإغلاق من <code>tagName</code> لأن <code>endTagName</code> يحمل رقم السطر ملحقاً به.
Jinja expression	يُقيّم التعبير مقابل ال Scope ويُستبدل بقيمته النصية، مع ترك محارف HTML الخاصة كما يفعل Jinja.

Jinja for	يُقيّم المُكْرَّر، ويُولّد الجسم مرة لكل عنصر في نطاق ابن جديد، مع دعم loop.index و loop.first و loop.last.
Jinja if	يُقيّم الشرط ويُولّد فرع واحد فقط. والفرق الجوهرى أن for تكرر بينما if اختيار.
Jinja comment	يُحذف من الخرج.

معالجة Jinja داخل سمات HTML

قيمة السمة في الشجرة نصّ خام وليست شجرة فرعية، فسمّة مثل الرابط الذي يحوي تعبير Jinja تُخزّن كنصّ كامل. لذلك أُضيفت دالة interpolate تبحث عن التعابير داخل النص، وتقيم مسار التعبير، وتستبدله بقيمته، ثم يمرّ الناتج على إعادة كتابة المسارات. قواعد هامة:

في Java لا تصلح القائمة شرطاً، بينما في Python و Jinja تُعدّ القائمة الفارغة قيمة كاذبة. لذلك نُقلت قواعد Python حرفياً: القيمة العدمية والصفر والنصّ الفارغ والقائمة الفارغة والقاموس الفارغ كلّها كاذبة. وبدون هذه القاعدة يفشل شرط if على القائمة دائماً.

4.5 إعادة التوليد عند تغيير البيانات

قد نُفّذ ذلك بطريقتين:

الطريقة	الآلية
مراقبة الملفات	يعمل المترجم بوضع watch فيراقب ملف app.py ومجلدي القوالب والملفات المرافقة. وعند حفظ أي ملف مصدري يُعاد التوليد كاملاً خلال أقل من مئتي ميلي ثانية. والمستمع هنا هو الجافا وليس التطبيق.
سيرفر المعاينة	يعمل المترجم بوضع serve فيخدم الصفحات المولّدة ويستقبل نماذجها. وعند إضافة منتج أو حذفه تُعدّل البيانات، ويُعاد التوليد فوراً، ثم يُعاد توجيه المتصفح إلى صفحة أنتجها المترجم لتوّه.

وفي الحالتين، إذا فشل تحليل ملف مصدري فلا يُولّد شيء ويبقى الخرج السابق كما هو، فالمترجم لا يُنتج مخرجات من ملف لم ينجح تحليله.

يفصل المشروع بين نوعين من المخرجات: الموقع الناتج، وتقارير مراحل الترجمة.

مجلد output وهو الموقع الناتج

الملف	المصدر
products.html	مُولّد من المسار الجذر
add_product.html	مُولّد من مسار إضافة منتج
product_details.html	مُولّد من مسار تفاصيل المنتج
static/	منسوخ كما هو: ملفات CSS والصور وملفات JavaScript
app.py	منسوخ كما هو، ولا يدخل في التحليل ولا التوليد

مجلد compiler_output وهو تقارير المراحل

الملف	المحتوى
ast_python.json	شجرة Python كاملة بصيغة JSON، وكل عقدة باسم صنفها ورقم سطرها وحقولها.
ast_jinja.json	أشجار القوالب في كائن واحد مفاتيحه أسماء ملفات القوالب.
semantic_report.txt	كل النتائج الدلالية من الجانبين مع عدّ الأخطاء والتحذيرات.
generation_log.txt	سجلّ التوليد: تقييم المتغيّرات، والمسارات المكتشفة، وتوسيع الحلقات، وإعادة كتابة الروابط.
symbol_table.txt	جداول الرموز: جدول Python وجداول HTML و CSS و Jinja لكل قالب.

6: نتائج التشغيل الفعلي

فيما يلي مخرجات تشغيل المترجم على مشروع المتجر المرفق:

[2] CODE GENERATION - PYTHON SIDE

```
module variables : [app, products, last_id]
URL parameters  : {product_id=[1, 2, 3]}
render plan     : 3 page(s)
```

[4] CODE GENERATION - JINJA SIDE

```
output/products.html (2805 chars, 0 warning(s))
```

```
output/add_product.html      (1838 chars, 0 warning(s))
output/product_details.html  (1521 chars, 0 warning(s))
```

[6] COMPILER OUTPUT

```
compiler_output/ast_python.json      (80188 bytes)
compiler_output/ast_jinja.json       (71017 bytes)
compiler_output/semantic_report.txt  (5843 bytes)
compiler_output/generation_log.txt   (1323 bytes)
compiler_output/symbol_table.txt     (5278 bytes)
```

مقتطف من سجلّ التوليد

```
[GEN] building the module context from the Python AST
[GEN] module context ready: [app, products, last_id]
[GEN] route / -> index
[GEN]   products.html (line 35) args={} ctx=[products]
[GEN] route /product/<int:product_id> -> product_details [product_id]
[GEN]   product_details.html (line 72) args={} ctx=[products]
[HTML] output/products.html
[HTML]   line 28: expanded for-loop 3 time(s)
[HTML]   line 39: route /product/1 -> product_details.html?product_id=1
```

خلاصة

يقرأ المترجم مشروع Flask كاملاً، ويحلّله نحويّاً ودلاليّاً على الجانبين، ثم يحسب القيم الحقيقية من شيفرة Python، ويُعدّ كل صفحة يمكن للتطبيق أن يخدمها، ويولّد لها HTML ثابتاً مرتبطاً ببعضه ببعض بروابط عاملة. ولا يُكتب داخل المترجم اسم قالب ولا اسم متغيّر ولا مسار، فكل ذلك مشتقّ من ملف app.py ومن البيانات التي يصرّح بها، وهو ما يجعل المترجم قابلاً للتطبيق على أي مشروع آخر لا على هذا المشروع وحده.

النهاية